Brain Tumor Detection with MRI/CT images using CNNs

Introduction

This project focused on the detection of brain tumors using Convolutional Neural Networks (CNNs). Brain tumors are a serious health concern, and identifying them early and correctly is essential for successful treatment and better chances of recovery for patients.

This project explores the application of CNNs, a powerful deep learning technique, to automatically identify and classify brain tumors from medical imaging data, specifically MRI scans. We will look into the challenges associated with brain tumor detection, the advantages of using CNNs for this task, and the overall methodology employed in this project. This introduction will lay the groundwork for understanding the subsequent stages of the project, including data preprocessing, model development, training, and evaluation.

Challenges in Brain Tumor Detection

Brain tumors are unusual clumps of cells that grow inside the brain. These growths can either be harmless (benign) or cancerous (malignant). How dangerous a tumor is depends on where it is in the brain, how big it is, and what kind of cells it's made of. Doctors usually find brain tumors by looking at medical images, like scans, and checking them by hand. This job is mostly done by radiologists. However, this way of finding tumors takes a lot of time, and it can be different depending on who is looking at the images. Also, people can make mistakes.

The subtle variations in image characteristics that differentiate between tumor types and healthy tissue can be challenging to discern, even for experienced professionals. Early detection can increase the survival rate.

Objective

To design, implement, and evaluate an end-to-end brain tumor detection system using a CNN that classifies MRI images into tumor vs healthy(binary classification).

- ***Data Acquisition and Preprocessing:*** Gather a comprehensive dataset of brain MRI scans, including images with and without tumors, and preprocess the data to ensure consistency and quality. This involves tasks such as image resizing, normalization, and noise reduction.
- ***CNN Model Development:*** Design and implement a CNN architecture suitable for brain tumor detection. This includes selecting appropriate layers, activation functions, and optimization algorithms.
- ***Model Training and Validation:*** The pre-processed MRI data will be used for training the CNN model, while its validation will be performed with separate dataset to ensure its generalization.
- ***Performance Evaluation:*** The performance of the trained CNN model is evaluated by considering metrics such as accuracy, precision, recall, and F1-score.

- ***Visualization and Interpretation:*** Visualize the features learned by the CNN model to gain insights into the underlying patterns that contribute to tumor detection.

Why Convolutional Neural Network?

Convolutional Neural Networks (CNNs) is a powerful model for image recognition and classification tasks.

Another important capability of CNNs involves complex feature learning from raw image data, which is especially valuable in the area of medical image analysis. Such networks can be trained to identify patterns and anomalies in MRI scans that are indicative of brain tumors with the goal of improving the accuracy and efficiency of diagnosis.

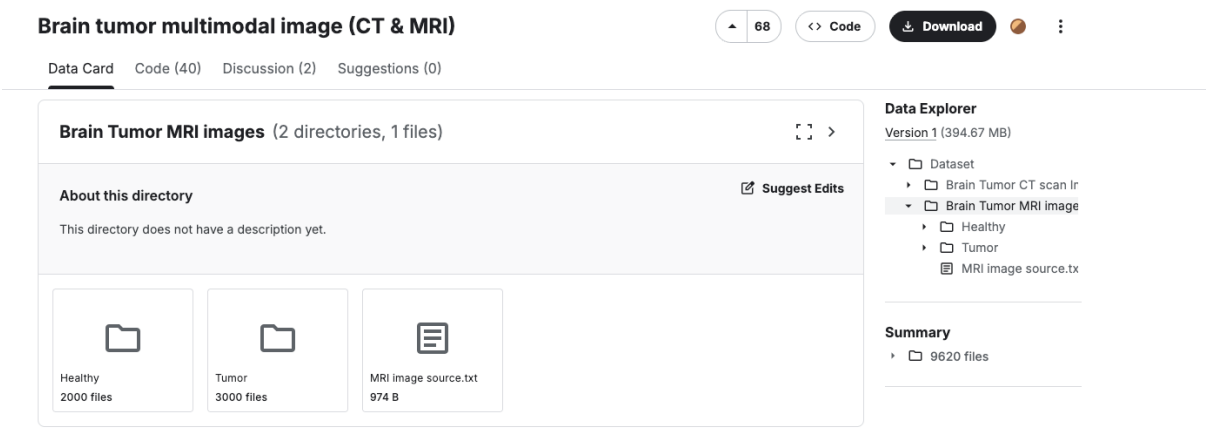
Dataset Overview:

Dataset used in this project was leveraged from one of the Kaggle open source dataset (https://www.kaggle.com/datasets/) which contains both MRI and CT scan images of brain in different layouts, size and orientation. The images in this dataset have been classified into two major class "Tumor" and "Healthy". In both classes we have sufficient amount of images equally distributed, so there is no data imbalance in this dataset.

- ***Computed Tomography (CT):*** CT scans utilize X-rays to produce detailed cross-sectional images of the brain. They are of particular help when it comes to the visualization of bone structures, calcification, or hemorrhages.
- ***Magnetic Resonance Imaging (MRI):*** MRI scans use magnetic fields and radio waves to produce high-resolution images of the brain's soft tissues. Different MRI sequences such as T1-weighted, T2-weighted, and FLAIR provide various contrasts, emphasizing various tissue characteristics and pathologies.

Data Size and Volume:

The size of the dataset is a significant factor to consider. A larger dataset generally allows for the training of more complex and accurate models. The Kaggle dataset used in this project contains 2000 images of healthy brain MRI and 3000 images of Tumor files which comprises of around 400MB of data.



Loading and Preprocessing the data:

Data Preprocessing:

MRI images often require preprocessing steps such as noise reduction, bias field correction, and normalization to improve the performance of algorithms.

Data Augmentation:

Due to the limited size of some brain tumor datasets, data augmentation techniques such as *rotation, scaling, flipping* can be used to increase the diversity of the training data and improve the generalization ability of models.

The ***"load_images_from_folder"*** function is responsible for loading and preprocessing MRI images from a project directory containing two folders—*healthy* and *tumor*.

For each folder, the function loads every image using OpenCV, converts it from BGR to RGB format, resizes it to a fixed **224×224 resolution**, and normalizes pixel values to the range 0–1. During this process, unreadable or corrupted images are safely skipped. Each image is then assigned a label-0 for healthy and 1 for tumor-based on the folder it came from. After processing all images, the function converts the collected images and labels into NumPy arrays and prints a summary showing the total number of images loaded, their shape, and the class distribution. Finally, it returns the processed image dataset and corresponding labels ready for model training.

```python
def load_images_from_folder(data_path, img_size=(224, 224)):
    images = []
    labels = []

    # Get actual folder names (case-insensitive)
    available_folders = os.listdir(data_path)

    # Find the correct folder names (case-insensitive matching)
    healthy_folder = None
    tumor_folder = None

    for folder in available_folders:
        if folder.lower() == 'healthy':
            healthy_folder = folder
        elif folder.lower() == 'tumor':
            tumor_folder = folder

    if not healthy_folder or not tumor_folder:
        raise ValueError(f"Could not find 'healthy' and 'tumor' folders in {data_path}. "
                         f"Available folders: {available_folders}")

    # Define class labels with actual folder names
    classes = [healthy_folder, tumor_folder]
    class_mapping = {healthy_folder: 0, tumor_folder: 1}

    for class_name in classes:
        class_path = os.path.join(data_path, class_name)

        if not os.path.exists(class_path):
            print(f"Warning: Folder {class_path} does not exist!")
            continue

        # Load all images from the class folder
        for img_name in os.listdir(class_path):
            img_path = os.path.join(class_path, img_name)

            try:
                # Read image
                img = cv2.imread(img_path)

                if img is None:
                    continue

                # Convert BGR to RGB (OpenCV loads as BGR)
                img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

                # Resize image to target size
                img = cv2.resize(img, img_size)

                # Normalize pixel values to [0, 1]`
                img = img / 255.0
```

```
============================================================

[1] Loading images from folder...
Loaded 5000 images
Image shape: (224, 224, 3)
Class distribution: Healthy=2000, Tumor=3000
```

Train, Test and Validation Split:

The *"**preprocess_data**"* function splits the loaded images and labels into training, validation, and test datasets using a two-stage stratified splitting process. First, it separates a portion of the data as the test set while maintaining the class balance. Then, it takes the remaining data and splits it again to create the validation set, adjusting the validation size proportionally to ensure the final split percentages are correct. After splitting, the function prints the number of images in each dataset and returns the *training, validation*, and *test subsets* along with their corresponding *labels*.

```python
def preprocess_data(images, labels, test_size=0.2, val_size=0.1):
    """
    Split data into train, validation, and test sets
    """
    # First split: separate test set
    X_temp, X_test, y_temp, y_test = train_test_split(
        images, labels, test_size=test_size, random_state=42, stratify=labels
    )

    # Second split: separate validation set from training
    val_size_adjusted = val_size / (1 - test_size)
    X_train, X_val, y_train, y_val = train_test_split(
        X_temp, y_temp, test_size=val_size_adjusted, random_state=42, stratify=y_temp
    )

    print(f"\nData split:")
    print(f"Training set: {len(X_train)} images")
    print(f"Validation set: {len(X_val)} images")
    print(f"Test set: {len(X_test)} images")

    return X_train, X_val, X_test, y_train, y_val, y_test
```

```
[2] Preprocessing and splitting data...

Data split:
Training set: 3500 images
Validation set: 500 images
Test set: 1000 images
```

CNN Model

The "***build_cnn_model***" function constructs a Convolutional Neural Network (CNN) designed for binary image classification, specifically to distinguish between tumor and healthy MRI scans. (Goodfellow, 2016)

The model begins with an input layer that accepts images of size **224×224** with *three* color channels. It then uses three consecutive convolutional blocks, each consisting of a Conv2D *layer, batch normalization, max pooling, and dropout.*

The convolutional layers extract features such as edges, textures, and patterns by applying *32, 64, and 128 filters* respectively, each with a *3×3 kernel.* The ReLU activation function is used in these layers to introduce non-linearity and allowing the network to learn complex patterns.

Batch normalization stabilizes and speeds up training by normalizing layer outputs, while max pooling reduces spatial dimensions and helps the model focus on the most important features. Dropout is included after each block to prevent overfitting by randomly disabling a fraction of neurons during training.

After the convolutional feature extraction stage, the model flattens the output and then feeds it into a fully connected dense layer consisting of ***128 neurons***. This dense layer also uses ReLU activation to learn higher-order relationships from the features extracted, followed by batch normalization and a larger dropout rate for further regularization. Finally, the output layer consists of a single neuron with a sigmoid activation function, which provides a probability in the range **between 0 and 1**, suitable for ***binary classification***. The function returns the complete CNN model, ready for compilation and training.

```python
def build_cnn_model(input_shape=(224, 224, 3), num_classes=1):
    """
    Build a CNN model for binary classification (tumor vs healthy)
    Architecture includes:
    - Multiple convolutional blocks with batch normalization
    - Max pooling for downsampling
    - Dropout for regularization
    - Dense layers for classification
    """
    model = keras.Sequential([
        # Input layer
        layers.Input(shape=input_shape),

        # First Convolutional Block
        layers.Conv2D(32, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Second Convolutional Block
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Third Convolutional Block
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.BatchNormalization(),
        layers.MaxPooling2D((2, 2)),
        layers.Dropout(0.25),

        # Flatten and Dense Layers
        layers.Flatten(),
        layers.Dense(128, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(0.5),

        # Output layer (sigmoid for binary classification)
        layers.Dense(num_classes, activation='sigmoid')
    ])

    return model
```

```
[3] Building CNN model...

Model: "sequential_1"
```

Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 224, 224, 32)	896
batch_normalization_4 (BatchNormalization)	(None, 224, 224, 32)	128
max_pooling2d_3 (MaxPooling2D)	(None, 112, 112, 32)	0
dropout_4 (Dropout)	(None, 112, 112, 32)	0
conv2d_4 (Conv2D)	(None, 112, 112, 64)	18,496
batch_normalization_5 (BatchNormalization)	(None, 112, 112, 64)	256
max_pooling2d_4 (MaxPooling2D)	(None, 56, 56, 64)	0
dropout_5 (Dropout)	(None, 56, 56, 64)	0
conv2d_5 (Conv2D)	(None, 56, 56, 128)	73,856
batch_normalization_6 (BatchNormalization)	(None, 56, 56, 128)	512
max_pooling2d_5 (MaxPooling2D)	(None, 28, 28, 128)	0
dropout_6 (Dropout)	(None, 28, 28, 128)	0
flatten_1 (Flatten)	(None, 100352)	0
dense_2 (Dense)	(None, 128)	12,845,184
batch_normalization_7 (BatchNormalization)	(None, 128)	512
dropout_7 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 1)	129

```
Total params: 12,939,969 (49.36 MB)

Trainable params: 12,939,265 (49.36 MB)

Non-trainable params: 704 (2.75 KB)
```

After CNN,

Total parameters: 12,939,969

Trainable parameters: 12,939,265

Non-trainable parameters: 704

The majority of parameters come from the dense layer with 128 neurons, which alone has over 12 million parameters.

Model Training:

The ***"train_model"*** function trains the CNN model with a combination of data augmentation and performance-monitoring callbacks. First, it compiles the model using the Adam optimizer, binary cross-entropy loss, and evaluation metrics such as *accuracy, precision, and recall* useful in binary classification.

To improve the model's generalization, it applies data augmentation on the training set using transformations like rotation, shifting, zooming, shearing, and horizontal flipping. These augmentations help the model learn robust features by exposing it to slightly altered versions of the images.

The function also sets up two important callbacks: **EarlyStopping**, which halts training when the validation loss stops improving and restores the best weights, and **ReduceLROnPlateau**, which lowers the learning rate when the model stagnates, allowing it to escape local minima.

Finally, the function returns the trained model along with the training history, which can later be used for visualization and analysis.

Binary Cross Entropy

It is a loss function used in **binary classification problems**, where the goal is to classify inputs into one of two possible classes.

```
[4] Training model...

Starting model training...
/Library/Frameworks/Python.framework/Versions/3.13/lib/python3.13/site-packages/keras/src/trainers/data_adapters/py_dataset_adapter.py:121: UserWarning: Your `PyDataset` class should call `super().__init__(**kwargs)` in its constructor. `**kwargs` ca
  self._warn_if_super_not_called()
Epoch 1/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 126s 1s/step - accuracy: 0.7377 - loss: 0.6017 - precision_1: 0.7731 - recall_1: 0.7967 - val_accuracy: 0.4000 - val_loss: 18.6891 - val_precision_1: 0.0000e+00 - val_recall_1: 0.0000e+00 - learning_rate: 0.0010
Epoch 2/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 120s 1s/step - accuracy: 0.8111 - loss: 0.4397 - precision_1: 0.8299 - recall_1: 0.8619 - val_accuracy: 0.4000 - val_loss: 17.8544 - val_precision_1: 0.0000e+00 - val_recall_1: 0.0000e+00 - learning_rate: 0.0010
Epoch 3/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 116s 1s/step - accuracy: 0.8371 - loss: 0.3828 - precision_1: 0.8535 - recall_1: 0.8795 - val_accuracy: 0.4000 - val_loss: 14.5558 - val_precision_1: 0.0000e+00 - val_recall_1: 0.0000e+00 - learning_rate: 0.0010
Epoch 4/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 118s 1s/step - accuracy: 0.8634 - loss: 0.3266 - precision_1: 0.8783 - recall_1: 0.8967 - val_accuracy: 0.4100 - val_loss: 2.2406 - val_precision_1: 0.7778 - val_recall_1: 0.0233 - learning_rate: 0.0010
Epoch 5/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 116s 1s/step - accuracy: 0.8734 - loss: 0.2954 - precision_1: 0.8873 - recall_1: 0.9038 - val_accuracy: 0.4000 - val_loss: 4.5412 - val_precision_1: 0.0000e+00 - val_recall_1: 0.0000e+00 - learning_rate: 0.0010
Epoch 6/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 126s 1s/step - accuracy: 0.9023 - loss: 0.2624 - precision_1: 0.9170 - recall_1: 0.9205 - val_accuracy: 0.8300 - val_loss: 0.7367 - val_precision_1: 0.9216 - val_recall_1: 0.7833 - learning_rate: 0.0010
Epoch 7/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 122s 1s/step - accuracy: 0.8949 - loss: 0.2588 - precision_1: 0.9062 - recall_1: 0.9200 - val_accuracy: 0.4320 - val_loss: 2.5536 - val_precision_1: 0.8636 - val_recall_1: 0.0633 - learning_rate: 0.0010
Epoch 8/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 119s 1s/step - accuracy: 0.8974 - loss: 0.2445 - precision_1: 0.9062 - recall_1: 0.9248 - val_accuracy: 0.8780 - val_loss: 0.3174 - val_precision_1: 0.8567 - val_recall_1: 0.9567 - learning_rate: 0.0010
Epoch 9/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 126s 1s/step - accuracy: 0.9197 - loss: 0.2106 - precision_1: 0.9296 - recall_1: 0.9371 - val_accuracy: 0.9120 - val_loss: 0.2477 - val_precision_1: 0.9706 - val_recall_1: 0.8800 - learning_rate: 0.0010
Epoch 10/10
110/110 ━━━━━━━━━━━━━━━━━━━━ 125s 1s/step - accuracy: 0.9174 - loss: 0.2073 - precision_1: 0.9241 - recall_1: 0.9395 - val_accuracy: 0.9300 - val_loss: 0.2091 - val_precision_1: 0.9402 - val_recall_1: 0.9433 - learning_rate: 0.0010
Restoring model weights from the end of the best epoch: 10.
```

Summary of Model Training:

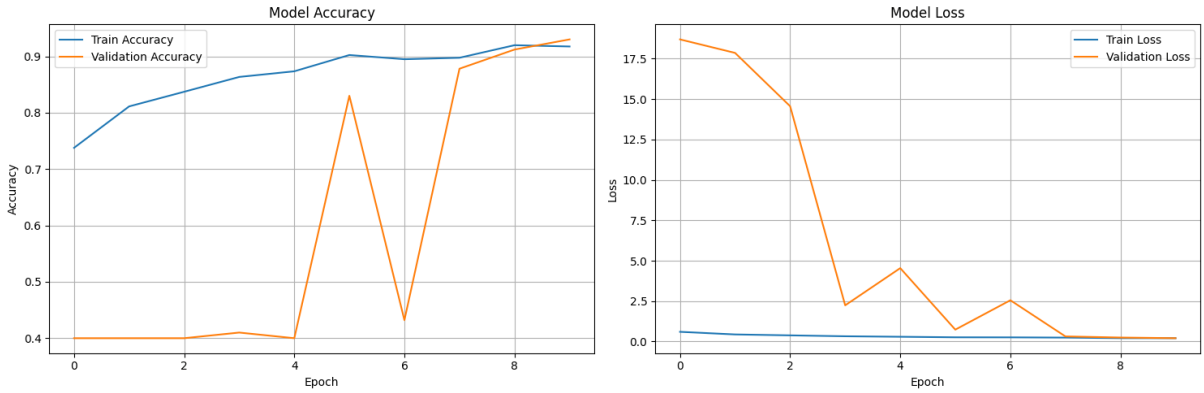
The above training log shows the CNN model being trained for 10 epochs, with performance steadily improving after each epoch. At the start of training, the accuracy was around **0.737** with a loss of **0.6017**, and both precision and recall were moderately low. As training progressed, the model consistently learned better feature representations, resulting in higher accuracy and lower loss values.

By the later epochs, the training accuracy increased to **over 0.91**, while the loss dropped significantly to around **0.21**, indicating strong learning and reduced error. Precision and recall also improved steadily, reaching values above **0.92** and **0.93**, showing that the model became better at correctly identifying both tumor and healthy cases.

Although the training metrics improved smoothly, the validation accuracy remained relatively low in the early epochs but gradually increased to **0.93** by epoch 10. The validation loss also decreased from very high values to **0.2091**, suggesting that the model eventually generalized well to unseen data.

EarlyStopping restored the ***best-performing*** weights at the end of training, ensuring the final model parameters correspond to the epoch with the optimal validation performance.

Model Evaluation on training/validation data:



The graphs illustrate how the model's accuracy and loss changed over the training epochs. The training accuracy steadily increased from around **0.74** in the first epoch to over **0.91** by the final epoch, showing consistent learning and improvement. Training loss followed a smooth downward trend, dropping from **about 0.6 to near zero**, which indicates that the model was able to ***fit the training data effectively***.

Validation performance, however, displayed more fluctuation. Validation accuracy stayed flat near **0.40** for several epochs, then jumped significantly in later epochs, eventually reaching values close to the training accuracy. Likewise, validation loss began *very high,* dropped sharply by the middle epochs, and eventually stabilized at a low value similar to the training loss.

This pattern suggests the model initially struggled to generalize but later adapted once it learned stronger features. The final convergence of both accuracy and loss for validation indicates *improved generalization* toward the end of training.

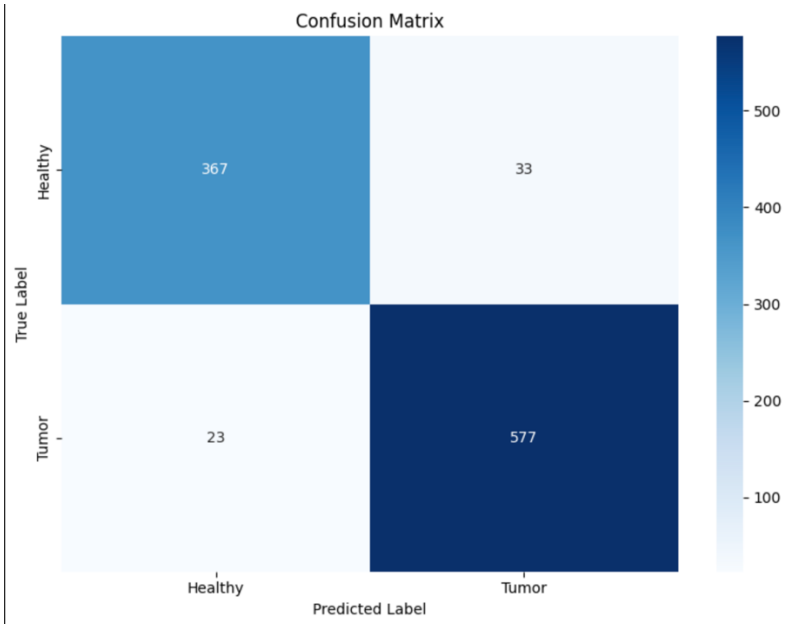
Model Evaluation on test data:

Model Metrics

```
============================================================
MODEL EVALUATION ON TEST SET
============================================================
32/32 ━━━━━━━━━━━━━━━━━━━━ 6s 196ms/step

Test Accuracy: 0.9440

Classification Report:
              precision    recall  f1-score   support

     Healthy       0.94      0.92      0.93       400
       Tumor       0.95      0.96      0.95       600

    accuracy                           0.94      1000
   macro avg       0.94      0.94      0.94      1000
weighted avg       0.94      0.94      0.94      1000
```

The model achieves a strong test *accuracy* of **94.4%**, showing that it generalizes well to unseen data. Both classes perform consistently well: 'healthy images' have a precision of **0.94** and *recall* of **0.92**, while 'tumor images' reach **0.95** *precision* and **0.96** *recall*. Overall, the model delivers ***balanced and reliable performance***, with macro and weighted averages also at **0.94**, indicating ***stable predictions*** across both categories.

Confusion Matrix

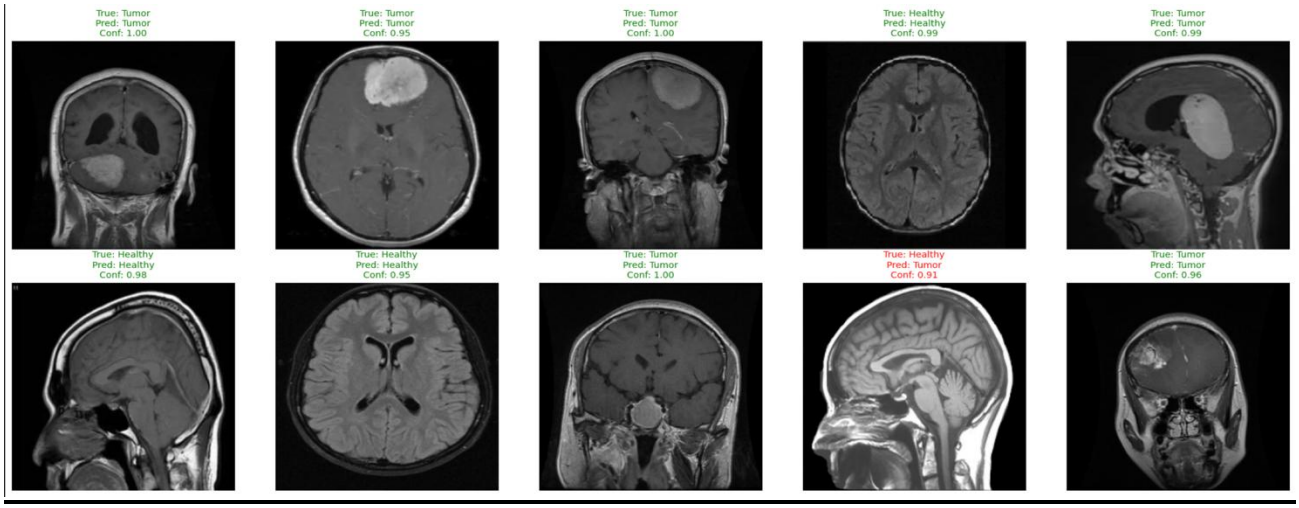


True Label	Predicted Healthy	Predicted Tumor	Explanation
Healthy	367 (True Negative)	33 (False Positive)	367 healthy images were correctly classified. 33 healthy images were incorrectly predicted as tumor.
Tumor	23 (False Negative)	577 (True Positive)	577 tumor images were correctly identified. 23 tumor images were incorrectly predicted as healthy.

The confusion matrix shows that the model correctly predicted **367 healthy** and **577 tumor** images. It misclassified **33 healthy** images as tumor and **23 tumor** images as healthy. Overall, the model demonstrates ***strong performance*** with very few incorrect predictions in both categories.

Sample Output:

Each Images were tagged with true and predicted value along with its confidence.



True Label, Predicted and Confidence in Tabular form:

Image No.	True Label	Predicted Label	Confidence	Interpretation
1	Tumor	Tumor	1	Correct – clear abnormality detected
2	Tumor	Tumor	0.95	Correct – high confidence
3	Tumor	Tumor	1	Correct – strong tumor features
4	Healthy	Healthy	0.99	Correct – normal brain tissue
5	Tumor	Tumor	0.99	Correct – visible mass
6	Healthy	Healthy	0.98	Correct – no lesion
7	Healthy	Healthy	0.95	Correct – subtle structures recognised
8	Tumor	Tumor	1	Correct – abnormal region identified
9	Healthy	Tumor	0.91	**Incorrect – false positive**
10	Tumor	Tumor	0.96	Correct – clear tumor region

Finally saved the model in h5 format for reusability.

Conclusion:

This project demonstrates the effectiveness of Convolutional Neural Networks (CNNs) in detecting brain tumors from MRI images. The model achieved strong performance with 94.4% accuracy and balanced precision and recall. The results highlight the potential of deep learning in medical imaging, enabling faster and more reliable diagnosis. Future improvements such as transfer learning and larger datasets can further enhance model performance.

Technologies Used:

- Python
- TensorFlow / Keras
- OpenCV
- NumPy, Pandas
- Matplotlib